LEDGERMIND — AI Settlement Intelligence Agent

Track: Razorpay Buildathon Track 04 — AI Finance Controller
Status: Implementation in progress
Architecture Version: 1.3 (Frozen)

The Problem

Reconciliation is still done by hand. A merchant receives a Razorpay settlement report, a bank statement, and a pile of transaction records — then spends hours matching UTRs, verifying fees, and hunting for missing refunds.

Most "AI reconciliation" tools dump raw CSV into an LLM and ask "does this match?" This is dangerous. The LLM hallucinates amounts, invents explanations, and auto-approves discrepancies it doesn't understand.

We built the opposite.

What LEDGERMIND Does

LEDGERMIND reconciles Razorpay settlements across four CSV sources:

- `transactions.csv` — captured payments
- `refunds.csv` — refund records

- `settlements.csv` — Razorpay settlement reports
- `bank_credits.csv` — bank statement credits

It processes a batch of 50+ settlements and reports:

- **Match rate** — how many settlements reconcile cleanly
- **Exception list** — what broke and why
- **Unresolved cases** — what the system honestly could not explain

---

The Core Idea

> **Deterministic when provable. AI when reasoning is required. Human when uncertainty remains.**

Three Lines of Defense

1. **Python Deterministic Engine** — Proves the math. Checks every reference, validates every fee, computes every expected amount. If it can explain the discrepancy with a rule, it does. No LLM involved.
2. **AI Investigator** — Steps in only when the math doesn't match and the rules can't explain why. It receives **structured evidence** (not raw CSV) and classifies the discrepancy. It **never** calculates, **never** approves, **never** invents records.

3. **Human Review Queue** — Every AI-investigated case routes here. The human reads the AI's explanation, checks the evidence, and clicks "Approve" or "Reject." The AI does not auto-approve anything. Ever.

Why This Is Not an LLM Wrapper

Wrapper Behavior	LEDGERMIND Behavior
Dumps raw CSV into LLM	Sends structured evidence packet with pre-computed amounts
LLM calculates `expected = payments - refunds`	Python calculates expected amount; LLM never sees raw numbers
LLM says "this looks fine" and approves	LLM classifies only; human must approve
LLM invents a refund to explain a gap	LLM citation validator rejects hallucinated evidence
One cherry-picked demo match	60-settlement batch with measured accuracy and honest exceptions

---

Safety Architecture

Five hard guarantees:

1. **AI never calculates money.** All arithmetic is done by Python. The LLM receives only pre-computed summaries.
2. **AI never modifies financial records.** The LLM output schema has no `amount`, `fee`, `refund`, or `tax` fields. `extra="forbid"` prevents injection.
3. **AI never auto-approves.** The `recommended_action` field is hardcoded to `ESCALATE_TO_HUMAN`. The system enforces `auto_approved_by_ai == 0` at the model level.
4. **Every AI claim must cite evidence.** The LLM must reference evidence IDs from the packet. Uncited claims are rejected with confidence 0.0.
5. **LLM failures fail safely.** Timeout, API error, rate limit, malformed JSON, hallucination — all route to `UNRESOLVED` + human review. No retries.

Architecture (High-Level)

```
CSV Upload → Validation → Normalization → Entity Linking
                    ↓
   Deterministic Reconciliation Engine
                    ↓
                    ┴
       ┌────────────┴────────────┐
       ↓                         ↓
  CLEAN_MATCH              MATH_DISCREPANCY
  (no discrepancy)         (all rules pass,
       ↓                   but math doesn't)
  Audit Trail                    ↓
       ↓              AI Investigator
```

```
Dashboard             (structured evidence)
                              ↓
                    ┌─────────┼─────────┐
                    ↓         ↓         ↓
              EXPLAINED  REVIEW   UNRESOLVED
                    │      REQUIRED      │
                    └─────────┼─────────┘
                              ↓
                     Human Review Queue
                              ↓
                       Audit Trail
                              ↓
                       Dashboard
```

Read the full architecture: architecture.md

Tech Stack

Layer	Technology
Backend	Python, FastAPI, Pydantic, Pandas
Database	SQLite (append-only audit log)
AI	LLM API (GPT-4o / Claude 3.5 Sonnet)
Frontend	Minimal React
Validation	Pydantic strict mode, integer paise only

How to Run

Prerequisites

```
python 3.11+
pip install -r requirements.txt
```

Generate Synthetic Data

```
python backend/generator.py --output data/demo/ --count 60
```

This creates 60 settlements with known ground truth: clean matches, missing references, bank mismatches, fee mismatches, timing issues, and unexplained gaps.

Run the Backend

```
uvicorn backend.main:app --reload
```

Upload and Process

1. Open the frontend at `http://localhost:5173`

2. Drag and drop the 4 CSV files from `data/demo/`
3. View the dashboard: match rate, exception list, unresolved cases
4. Click any settlement to see the full reconciliation trace
5. Review AI-investigated cases in the human queue

Run Evaluation

```
python -m pytest tests/test_e2e.py -v
```

This runs the full pipeline against ground truth and reports:

- Match rate
- False accept rate
- AI invocation rate
- Processing time per settlement

Demo Script (5 Minutes)

Time	What You Show
0:00	Upload 4 CSVs. Dashboard: "60 processed, 34 clean, 20 exceptions, 6 unresolved, **0 auto-approved**"
0:45	Click a **CLEAN_MATCH**. Show the reconciliation trace: every amount calculated by Python, no AI involved.
1:30	Click a **FEE_MISMATCH**. Show: "AI not required. Deterministic rule identified the exact violation."
2:00	Click a **REFUND_TIMING** case. Expand the AI evidence packet. Show the structured JSON sent to the LLM. Confidence: 0.82. Status: **REVIEW_REQUIRED**.
2:45	Click "Approve Explanation." Status changes to **EXPLAINED**. Audit trail records the human action.
3:15	Click the **UNRESOLVED** case (1,775 gap). AI: "INSUFFICIENT EVIDENCE. ESCALATED TO HUMAN REVIEW." Confidence: 0.15.
3:45	Show the **Safety Guarantees** panel. "AI never calculates, never approves, never invents evidence."
4:00	Show **Batch Patterns**: "3 settlements on Aug 20 show 1-paise fee mismatches. Suggest reviewing fee rounding rule."

Table 3 – continued

Time	What You Show
4:45	Show **Evaluation Metrics**: "Ground truth: 60 labeled. Match rate 71.7%. False accept 5.0%. **0 auto-approved.**"

Key Metrics (From Evaluation Harness)

Metric	Target	Why It Matters
Match Rate	~70%	Honest accuracy on synthetic ground truth
False Accept Rate	<5%	Critical safety metric —how many bad settlements were marked clean
AI Invocation Rate	~15%	Proves AI is used appropriately, not as a default
AI Auto-Approval Rate	**0%**	By design. AI never approves.
Processing Time	<1s/settlement	Throughput for 50+ record batches

---

What We Did NOT Build

- User authentication (single-user demo)
- PostgreSQL (SQLite is sufficient for 50+ records)
- Real-time webhooks (batch processing only)
- Email/Slack notifications (out of scope)
- PDF or bank statement OCR (CSV input only)
- Multi-currency (INR/paise only)
- Chart visualizations (tables are sufficient)
- LLM fine-tuning (prompt engineering only)
- Microservices, Docker, Kubernetes (flat Python files)

See full scope lock: architecture.md#scope-lock

Project Structure

```
ledgermind/
├── backend/
│   ├── main.py              # FastAPI endpoints
│   ├── models.py            # Pydantic data models (strict, validated)
│   ├── ingestion.py         # CSV validation + idempotency
│   ├── linking.py           # Entity linker
│   ├── engine.py            # Deterministic reconciliation
```

```
│   ├── ai_investigator.py   # LLM integration (constrained)
│   ├── batch_analyzer.py    # Cross-settlement pattern detection
│   ├── audit.py             # Append-only audit logger
│   └── generator.py         # Synthetic data with ground truth
├── frontend/                # Minimal React
├── tests/                   # Test-first checkpoints
├── architecture.md          # Full technical specification
└── README.md                # This file
```

Safety-First Design Philosophy

This project is built on a simple conviction: **in financial systems, the AI must know when it doesn't know.**

Most AI demos optimize for accuracy. We optimized for **honest uncertainty.** The system's proudest moment is not when it explains a discrepancy — it's when it **refuses to guess** and escalates to a human.

That refusal is not a bug. It's the feature.

License

MIT — Built for Razorpay Buildathon 2026.